

Especificación de la API REST

Sistema Polaflix

Sergio Lanza Teja

1 Introducción

Este documento detalla la especificación de la API REST desarrollada para la plataforma Polaflix. El diseño se ha pensado específicamente para dar servicio a la aplicación Frontend, siguiendo una separación clara de responsabilidades.

Los endpoints se centran exclusivamente en la experiencia del cliente: navegar por el catálogo, gestionar sus visualizaciones y consultar su progreso (aunque no esté implementado, se han dejado los endpoints necesarios para que el usuario pueda ver sus facturas). Las operaciones de administración (como añadir series nuevas o modificar facturas manualmente) no se exponen aquí, ya que pertenecen a un servicio interno de *Backoffice*.

2 Formato Estándar de Respuesta (Wrapper)

Para que el desarrollo del Frontend sea predecible y no haya sorpresas, absolutamente todas las respuestas de la API (tanto si van bien como si fallan) se envuelven en una clase contenedora genérica que he llamado `ApiResponse<T>`.

Estructura base de un JSON de éxito (Ejemplo 200 OK sacado de Swagger UI):

```
{
  "status": 200,
  "message": "Usuario obtenido exitosamente",
  "data": {
    "idUserario": 1,
    "nombreUsuario": "Paco",
    "cuentaBancaria": "ES111122223333",
    "suscripcion": {
      "idSuscripcion": 1,
      "tipo": "SuscripcionFija",
      "cuotaMensual": 20
    }
  },
  "timestamp": "2026-05-31T18:49:13.982876"
}
```

Dentro del campo `data` viaja el DTO correspondiente. Para no saturar la red enviando datos innecesarios, uso las anotaciones `@JsonView` de Jackson, filtrando dinámicamente qué campos se envían según el endpoint.

3 Gestión de Errores y Códigos HTTP

He centralizado el manejo de errores creando un `@RestControllerAdvice (GlobalExceptionHandler)`. Esta clase captura las excepciones de Java y las transforma en respuestas HTTP limpias. En caso de error, el campo `data` desaparece y se muestra un código descriptivo.

- **200 OK / 201 Created:** La petición se ha procesado correctamente.
- **400 Bad Request (VALIDATION_ERROR):** Salta cuando el cliente envía un JSON que no cumple las reglas del DTO (anotaciones `@Valid`). El sistema devuelve exactamente qué campo ha fallado (por ejemplo, si el IBAN tiene un formato incorrecto).
- **404 Not Found (USER_NOT_FOUND, SERIE_NOT_FOUND):** El ID solicitado no existe en la base de datos.
- **500 Internal Server Error (INTERNAL_SERVER_ERROR):** Excepciones genéricas no controladas.

4 Catálogo de Endpoints

4.1 Recurso: Usuarios (/usuarios)

Este es el controlador principal de la interacción del cliente. Gestiona su perfil, sus listas y su facturación. He decidido que las facturas cuelguen de aquí porque, por diseño de negocio, una factura pertenece al historial de un usuario concreto y no se accede a ella de forma aislada en la app pública.

- GET /usuarios/{id}
Recupera el perfil del usuario, incluyendo los detalles de su plan de suscripción.
- POST /usuarios/{usuarioId}/series/{serieId}
Añade manualmente una serie al listado de "Pendientes" del usuario.
- GET /usuarios/{usuarioId}/series/pendientes | empezadas | terminadas
Tres endpoints separados para recuperar de forma sencilla las listas de series del usuario.
- POST /usuarios/{usuarioId}/capitulos/visto
Endpoint principal de negocio. Registra que un usuario ha visto un capítulo usando parámetros por URL (`serieId`, `temp` y `cap`). Por debajo, el sistema actualiza su progreso, calcula si hay que cobrarle algo extra (si está en Bajo Demanda) y mueve la serie de lista automáticamente.
- GET /usuarios/{usuarioId}/progreso/{serieId}
Devuelve la última temporada y capítulo que el usuario ha visto de una serie concreta.
- GET /usuarios/{usuarioId}/facturas
Permite al usuario consultar su historial de facturas. Usa una vista básica del DTO para devolver la lista rápidamente.

4.2 Recurso: Catálogo de Series (/series)

Controlador de *solo lectura* que permite explorar el contenido de la plataforma.

- GET /series
Devuelve todo el catálogo de series. Usa una vista *Basic* para no sobrecargar el Frontend trayendo todos los capítulos de golpe.

- GET /series/{id}
Trae el detalle completo de una serie. Al usar la vista *Detailed*, aquí sí se inyecta todo el árbol de temporadas y capítulos.
- GET /series/buscar/titulo?titulo={texto}
Buscador simple para filtrar el catálogo enviando una cadena de texto.
- GET /series/{serieId}/temporadas
Permite cargar exclusivamente la lista de temporadas de una serie concreta.